

Operating System for AeroNav Tech's autonomous drone navigation system

u5602780

January 7, 2025

1 Introduction

This page complements the A3 poster by explaining why certain components of the OS for ADNS were chosen. References are included to clarify how these components work, supporting both research and the development of this operating system.

2 Process Management

Fork, Wait, Exec, and Exit. are UNIX based system calls they are time-tested and form the basis for many modern operating systems. It would be the best practise to use them for our OS. [1] pp. 68–72 (Fork), 119–123 (Wait, Exec, Exit).

It is important to describe how processes are created, terminated, and how they interact with each other. Inter-Process Communication (IPC) and process states serve this exact function. [1] pp. 105–110.

3 Process Scheduling

Microsoft's Real-Time Priority System Levels [2] provide an excellent example of assigning priority levels to different types of programmes based on their importance.

Using two separate types of process scheduling for critical and less critical tasks at different queue levels enhances the robustness of our OS. Critical tasks will utilise lottery scheduling [3] (Ch. 9). For example, a task responsible for keeping the drone in the air is allocated 50% more compute time than a task responsible for moving the drone to a set location. Both tasks are important, but their priorities differ.

Less critical tasks should use round-robin scheduling [3] (Ch. 7.7), which ensures fast response times, allowing more important tasks to be addressed more efficiently.

4 Memory Management

Address space defines how processes are allocated and managed in RAM [4] (Ch. 3.2). Unprotected memory spaces can result in significant security vulnerabilities. One effective mitigation is the use of polymorphic canaries [5], which detect buffer overflows and terminate compromised processes to prevent further damage.

Memory paging is a widely used technique for managing RAM. It allows multiple programmes to run concurrently by dividing memory into fixed-size blocks called pages. This approach minimises RAM wastage, improves overall efficiency, and is more suitable for systems with limited RAM compared to segmentation [4] (Ch. 3.3).

An additional benefit of paging is its support for data swapping with secondary storage. This reduces RAM requirements and prevents processes from crashing when the required memory exceeds the available physical RAM [4] (Ch. 3.2.2).

5 References

1. Silberschatz, A., Galvin, P. B., and Gagne, G. (2018). *Operating System Concepts*. 10th ed. Hoboken, N.J.: Wiley.
2. Archiveddocs (2012). Real-Time Priority System Levels (Windows Embedded CE 6.0). [online] learn.microsoft.com. Available at: [https://learn.microsoft.com/en-us/previous-versions/windows/embedded/ee483074\(v=winembedded.60\)](https://learn.microsoft.com/en-us/previous-versions/windows/embedded/ee483074(v=winembedded.60)).
3. Arpaci-Dusseau, R. H., and Arpaci-Dusseau, A. C. (2023). *Operating Systems: Three Easy Pieces*. Madison: Arpaci-Dusseau Books.
4. Tanenbaum, A. S., and Bos, H. (2014). *Modern Operating Systems*. 4th ed. Pearson.
5. Z. Wang, X. Ding, C. Pang, J. Guo, J. Zhu and B. Mao, "To Detect Stack Buffer Overflow with Polymorphic Canaries," 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) <https://ieeexplore.ieee.org/document/8416487>.